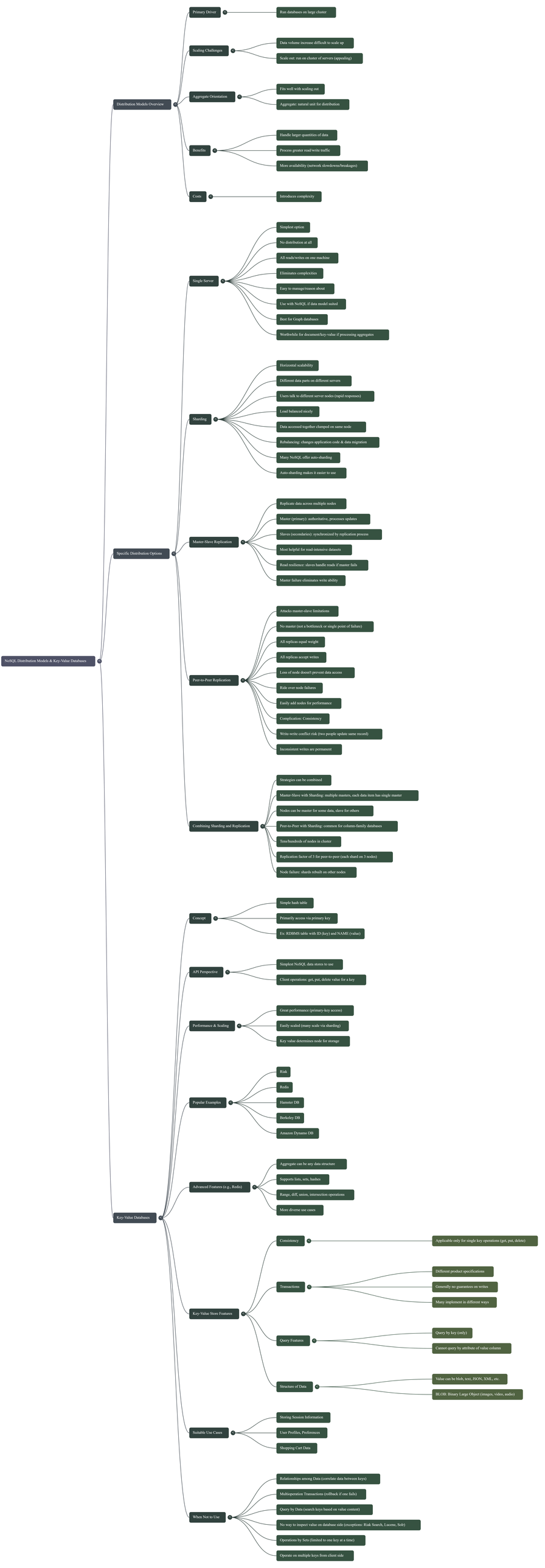
NoSQL Distribution Models & Key-Value Databases
Distribution Models Overview
Primary Driver
Run databases on large cluster
Scaling Challenges
Data volume increase difficult to scale up
Scale out: run on cluster of servers (appealing)
Aggregate Orientation
Fits well with scaling out
Aggregate: natural unit for distribution
Benefits
Handle larger quantities of data
Process greater read/write traffic
More availability (network slowdowns/breakages)
Costs
Introduces complexity
Specific Distribution Options
Single Server
Simplest option
No distribution at all
All reads/writes on one machine
Eliminates complexities
Easy to manage/reason about
Use with NoSQL if data model suited
Best for Graph databases
Worthwhile for document/key-value if processing aggregates
Sharding
Horizontal scalability
Different data parts on different servers
Users talk to different server nodes (rapid responses)
Load balanced nicely
Data accessed together clumped on same node
Rebalancing: changes application code & data migration
Many NoSQL offer auto-sharding
Auto-sharding makes it easier to use
Master-Slave Replication
Replicate data across multiple nodes
Master (primary): authoritative, processes updates
Slaves (secondaries): synchronized by replication process
Most helpful for read-intensive datasets
Read resilience: slaves handle reads if master fails
Master failure eliminates write ability
Peer-to-Peer Replication
Attacks master-slave limitations
No master (not a bottleneck or single point of failure)
All replicas equal weight
All replicas accept writes
Loss of node doesn't prevent data access
Ride over node failures
Easily add nodes for performance
Complication: Consistency
Write-write conflict risk (two people update same record)
Inconsistent writes are permanent
Combining Sharding and Replication
Strategies can be combined
Master-Slave with Sharding: multiple masters, each data item has single master
Nodes can be master for some data, slave for others
Peer-to-Peer with Sharding: common for column-family databases
Tens/hundreds of nodes in cluster
Replication factor of 3 for peer-to-peer (each shard on 3 nodes)
Node failure: shards rebuilt on other nodes
Key-Value Databases
Concept
Simple hash table
Primarily access via primary key
Ex: RDBMS table with ID (key) and NAME (value)
API Perspective
Simplest NoSQL data stores to use
Client operations: get, put, delete value for a key
Performance & Scaling
Great performance (primary-key access)
Easily scaled (many scale via sharding)
Key value determines node for storage
Popular Examples
Riak
Redis
Hamster DB
Berkeley DB
Amazon Dynamo DB
Advanced Features (e.g., Redis)
Aggregate can be any data structure
Supports lists, sets, hashes
Range, diff, union, intersection operations
More diverse use cases
Key-Value Store Features
Consistency
Applicable only for single key operations (get, put, delete)
Transactions
Different product specifications
Generally no guarantees on writes
Many implement in different ways
Query Features
Query by key (only)
Cannot query by attribute of value column
Structure of Data
Value can be blob, text, JSON, XML, etc.
BLOB: Binary Large Object (images, video, audio)
Suitable Use Cases
Storing Session Information
User Profiles, Preferences
Shopping Cart Data
When Not to Use
Relationships among Data (correlate data between keys)
Multioperation Transactions (rollback if one fails)
Query by Data (search keys based on value content)
No way to inspect value on database side (exceptions: Riak Search, Lucene, Solr)
Operations by Sets (limited to one key at a time)
Operate on multiple keys from client side